USER GUIDE

Data Quality Scanner V2.5

Pre-Migration Data Quality Assessment & Migration-Readiness Decision Platform

Intended Audience: Data Analysts, Migration Teams, Data Engineers

1. Introduction & Purpose

Data Quality Scanner is an automated pre-migration data quality assessment platform that profiles source datasets, infers high-confidence business rules, detects quality issues, converts them into a migration-readiness decision, and provides actionable record-level reports before migration.

What Problem It Solves

Data migrations often encounter costly roadblocks when legacy data contains hidden defects—such as missing values in mandatory columns, duplicate record keys, invalid dates, broken foreign-key relationships, and invisible whitespace formatting errors.

Data Quality Scanner catches these defects *before* migration scripts execute or data loads into target systems, helping teams identify and address issues earlier in the migration process.

When to Use It

- **Discovery Phase:** Profile legacy source extracts to understand data cleanliness and structure.
- **ETL Development:** Validate transformed datasets against required business constraints.
- **Pre-Go-Live Dress Rehearsals:** Verify final extract readiness and generate reports for migration review and remediation.

What Users Receive After a Scan

🎯 Overall Readiness Score (0–100)

An objective score and status (*Ready*, *Minor Cleansing*, *Needs Cleansing*, *Not Ready*) showing if data is fit for migration.

📊 Diagnostic Dimension Meters

Health ratings across six core quality dimensions: Completeness, Uniqueness, Validity, Consistency, Integrity, and Cleanliness.

🔍 Record-Level Issue Logs

Exact row numbers, column names, actual defective values, and error explanations for direct remediation.

📄 Stakeholder Deliverables

Detailed multi-sheet Excel (`.xlsx`), interactive HTML, and machine-readable JSON assessment reports.

2. Which Mode Should I Use?

The application offers two execution modes that share the exact same core calculation engine:

🖥️ Desktop GUI Mode

Best for interactive exploration and visual analysis.

- Visual dataset upload with 10-row preview grid.

⚡ Headless Batch / CLI Mode

Best for automated, scheduled, or multi-file runs.

- Interactive confirmation dialog for inferred mandatory fields.
- Circular readiness score gauge and dynamic dimension meters.
- Filterable issues table with real-time text search.
- Visual parent-child relationship diagrams and orphan rows.
- Double-click to reload historical scans from SQLite.

Launch command: `python main.py`

- Runs directly in terminals, shell scripts, or CI/CD pipelines.
- Zero graphical or display-server dependencies.
- Deterministic process exit codes (`0` , `1` , `2` , `3`) for build pipelines.
- Direct flags for automated report generation (`--excel` , `--html` , `--json`).
- Specific dataset profile and Excel sheet selection arguments.

Launch command: `python batch_runner.py [options] DATASET...`

3. System Requirements & Important Folders

System Requirements

Requirement	Specification	Type
Operating System	Validated Environment: Windows 10/11. *Other operating systems may work where all project dependencies are supported, but should be validated before deployment.*	Required
Python Environment	Python 3.11 or later (64-bit)	Required
Dependencies	Installed via `requirements.txt` (PySide6, pandas, openpyxl, PyYAML, Jinja2, matplotlib, charset-normalizer)	Required
File Permissions	Read access to input datasets; write access to `data/` (database) and `reports/` (outputs)	Required
Test Dependencies	`pytest` and `pytest-qt` (installed via `requirements-dev.txt`)	Optional

Important Project Folders

Folder	Purpose
`config/`	Business rules (`config.yaml`) and admin technical parameters (`admin_config.yaml`).
`assets/branding/`	Company branding assets displayed in application deliverables.
`reports/`	Default destination folder for generated Excel, HTML, and JSON assessment reports.
`data/`	Local SQLite database (`scanner.db`) storing historical assessment metadata.
`samples/`	Pre-packaged sample datasets provided for immediate testing and demonstration.

4. Input Data Requirements

Supported File Formats

- **CSV Files (`.csv`):** Delimiters (comma, semicolon, tab, pipe) and text encodings (UTF-8, Latin-1, Windows-1252) are detected automatically.
- **Excel Workbooks (`.xlsx` , `.xls`):** Single-sheet and multi-sheet workbooks are supported. The user can select the target worksheet during upload or specify it in CLI mode.

Structural Expectations

- **Header Row:** Row 1 must contain column names. Column names are automatically trimmed of extra spaces upon loading.
- **Tabular Consistency:** Data rows must follow a consistent column structure. Empty lines are handled safely.
- **Multi-Dataset Analysis:** When checking foreign-key relationships, load both the parent dataset (e.g. Customers) and child dataset (e.g. Orders) in the same session.

Handled Data Conditions

Condition	Scanner Handling
Blanks, Nulls, & Whitespace	Differentiates true SQL nulls, empty strings (`""`), and whitespace strings (`" "`).
Corrupted or Locked Files	Displays a friendly notification explaining that the file cannot be accessed or is open in another program.
Broken References	Cross-matches child foreign keys against parent keys, listing exact row numbers and missing key values.
Large Datasets (>500k rows)	A non-blocking notification reminds users that deep pattern scanning may require extra memory.

5. Desktop GUI Mode (Setup & Walkthrough)

Follow these steps to set up the environment, launch the application, and complete an end-to-end data quality scan:

Step 1 — Open Terminal & Navigate to Project Root

Open your command-line terminal and change directory to the project root folder containing `main.py` and `batch_runner.py` . You must first move into the project directory because the virtual environment should be created inside the project:

```
cd path/to/migration-dq-scanner
```

Step 2 — Create the Python Virtual Environment

Create an isolated Python virtual environment inside the project directory (normally only needs to be done once during initial setup):

```
python -m venv .venv
```

This creates the isolated `.venv` environment inside the project directory to ensure project dependencies do not conflict with your system Python packages.

Step 3 — Activate the Virtual Environment

Activate the virtual environment corresponding to your operating system shell:

```
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1

# Windows Command Prompt (CMD):
.venv\Scripts\activate.bat

# Linux / macOS:
source .venv/bin/activate
```

Upon activation, your shell prompt will show the `(.venv)` prefix indicating that the isolated environment is active.

Step 4 — Install Project Dependencies

Install all required application libraries using pip:

```
python -m pip install -r requirements.txt
```

This installs PySide6, pandas, openpyxl, PyYAML, Jinja2, matplotlib, and charset-normalizer as specified in the project requirements.

Step 5 — Start the Application

Launch the Desktop GUI by running:

```
python main.py
```

The main application window opens displaying the header and the Dataset Upload section.



Figure 5.1: Top application header displayed upon launch.

Step 6 — Add Datasets

In the **Dataset Upload** section:

- Click ➕ **Add CSV File** to browse and select a delimited text file.
- Click ➕ **Add Excel Workbook** to select a spreadsheet.
- Use 🗑️ **Remove Selected** if you need to remove a dataset from the queue.

Step 7 — Select Excel Worksheet (If Applicable)

When loading a multi-sheet Excel workbook, a sheet selection modal dialog opens automatically prompting you to choose which worksheet to analyze. Select the appropriate sheet and click **Select**.

Step 8 — Review Dataset Preview

Click on any dataset in the **Loaded Datasets** list. The **Dataset Preview** grid immediately displays the first 10 rows of the selected dataset alongside total row and column counts.

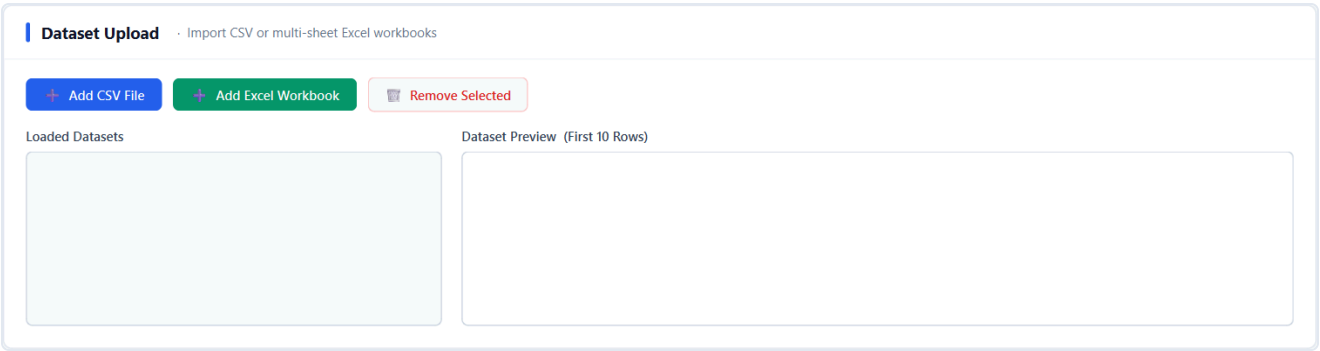


Figure 5.2: Loaded datasets list and first 10 rows preview grid.

Step 9 — Run Analysis & Confirm Inferred Rules

Click the green 🚀 **Run Data Quality Analysis** button.

If the dataset contains columns that strongly resemble mandatory business fields (not already defined in configuration), the scanner opens the **Mandatory Fields Detected** review dialog:

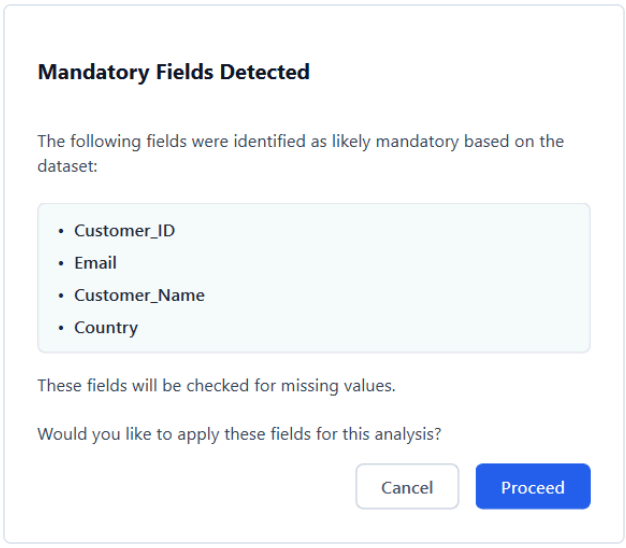


Figure 5.3: Review dialog allowing users to confirm or decline inferred mandatory fields.

- **Proceed (Default):** Applies these fields as mandatory requirements for this analysis run.
- **Cancel:** Aborts analysis execution cleanly without altering settings.

Step 10 — Review Results, Dimensions, & Issues

When analysis finishes, the dashboard automatically scrolls down to display findings:

- **Overall Data Quality Result:** Circular gauge showing the Readiness Score (0–100), status, and issue counts.
- **Quality Dimension Scores:** Six diagnostic health progress bars.
- **Quality Issues Table:** Filterable table showing severity, category, dataset, column, quality dimension, and issue description.
- **Relationship Integrity:** Parent-child linkage diagram, pass/fail badge, and orphan rows table.

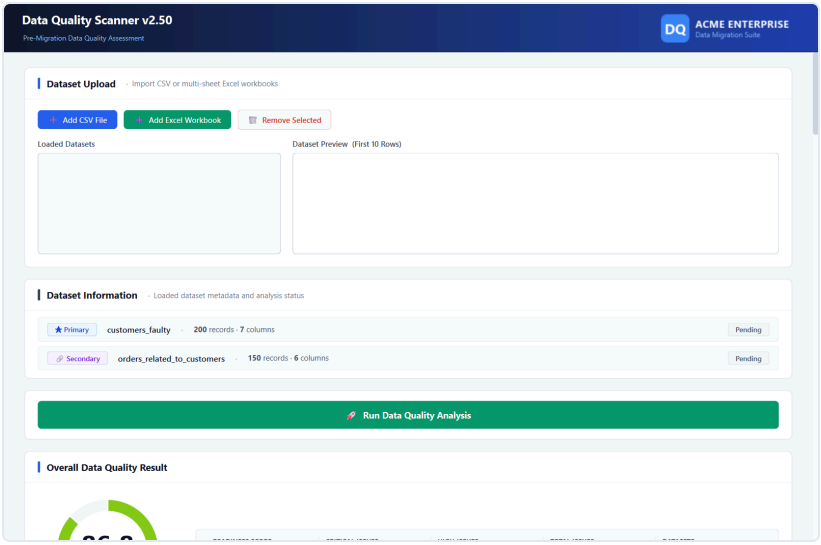


Figure 5.4: Live assessment dashboard displaying scores, dimensions, and issues.

Step 11 — Review Recommendations

Scroll to the **Recommendations** card to review prioritized remediation actions sorted Critical-first.

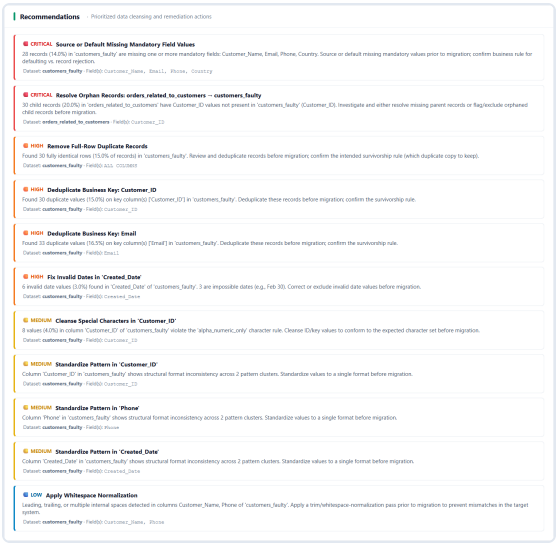


Figure 5.5: Prioritized remediation recommendations.

Step 12 — Generate Reports

In the **Export Reports** section, check your desired formats (*HTML Report*, *Excel Workbook*, *JSON*), choose an output directory if needed, and click 📥 **Generate & Save Reports**.

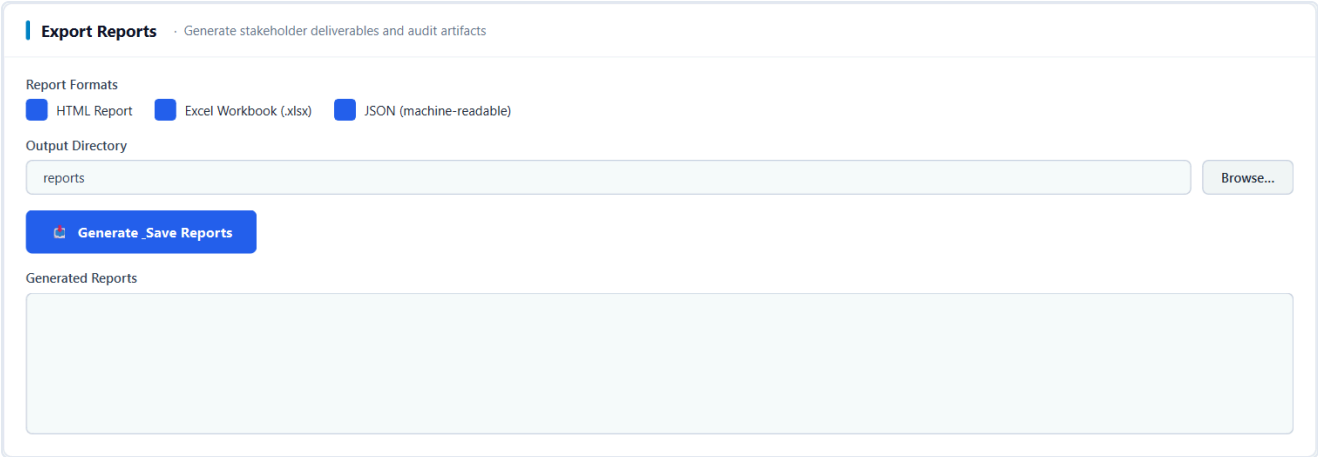


Figure 5.6: Selecting report formats and destination folder.

Step 13 — Review Scan History

By default, completed scans are recorded in the local SQLite database (`data/scanner.db`). Persistence can be disabled in Batch/CLI mode using `--no-db` or `--skip-db`. Review past assessments in the **Scan History** table. Double-clicking any past scan row reloads its scores, charts, and findings directly back into the dashboard.

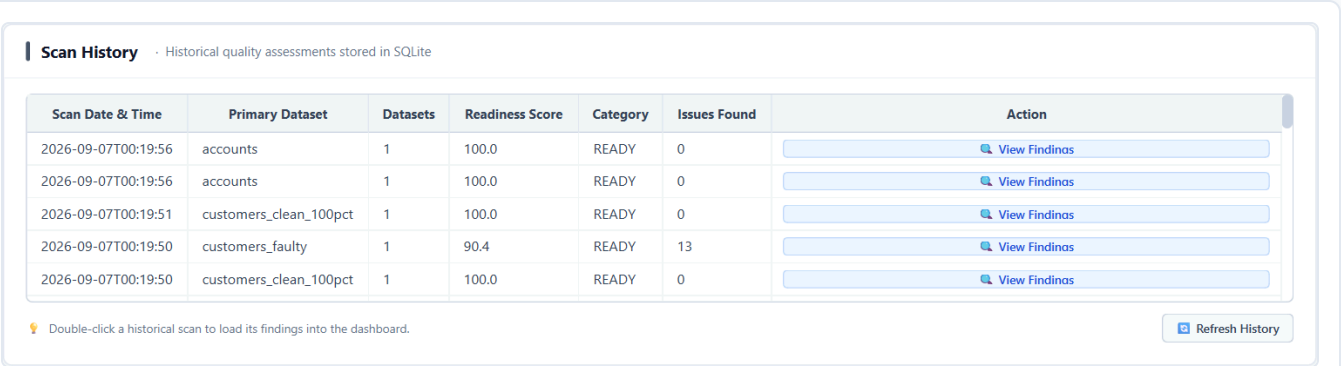


Figure 5.7: Historical scan log with double-click reload functionality.

6. The 8 Quality Checks

The engine evaluates source data across eight standardized quality checks:

Check	What It Checks	Why It Matters for Migration
1. Duplicates	Identifies full-row identical records and duplicate business keys (e.g. duplicate Customer ID).	Prevents primary key collisions and unique constraint rejections during target database loading.
2. Nulls & Blanks	Separates true nulls, empty strings, and whitespace-only values across all columns.	Identifies unpopulated fields and differentiates data gaps from formatting defects.
3. Mandatory Fields	Verifies that required business fields have non-empty data in every row.	Catches records that would violate target system `NOT NULL` constraints.
4. Whitespace	Detects leading spaces, trailing spaces, and multiple consecutive internal spaces.	Invisible whitespace causes failed record lookups, broken text joins, and distorted reporting.
5. Special Characters	Scans for global anomaly characters (`$` , `@` , `#` , `%` , `&` , `*`) and preset format rules.	Prevents parser errors, SQL/XML syntax breaks, and illegal characters in strict target fields.
6. Pattern Consistency	Converts values to structural masks (e.g. `AAA-999`) and clusters dominant vs outlier patterns.	Surfaces legacy entry variations and non-standard coding formats.
7. Dates	Validates dates against accepted formats and flags unparseable or mixed date representations.	Eliminates date conversion crashes and ambiguous day/month transpositions during ETL.
8. Relationships	Cross-matches child foreign keys against parent primary keys to detect missing parents.	Prevents orphan child records that violate relational database foreign-key integrity.

7. Understanding the Readiness Score

> **✓ 100% Clean Baseline Guarantee**
>
> A clean dataset with zero defects receives an exact score of **100.0**. Points are deducted only when genuine issues are detected.

How the Score is Calculated

The readiness score is calculated through a simple, transparent flow:

Detected Issues | **Severity Weight** | **Affected Record Volume** | **Score Deduction**

Deductions start with the severity weight of the defect and scale based on what percentage of the dataset is affected. Higher severity issues and larger volumes of bad data produce larger score reductions.

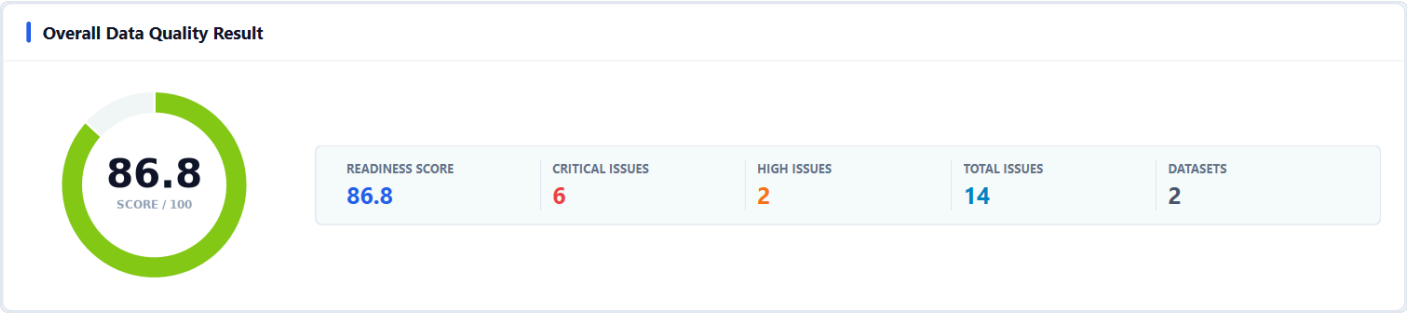


Figure 7.1: Circular readiness score gauge and issue severity distribution.

Severity Levels

Severity	Base Impact	Types of Issues
CRITICAL	15.0 pts max	Migration blockers: missing mandatory fields, duplicate unique keys, orphan foreign keys.
HIGH	8.0 pts max	Severe anomalies: unparseable dates, high null rates (≥20%), high duplicate rates (≥5%).
MEDIUM	3.0 pts max	Moderate anomalies: special character violations, moderate null/duplicate rates.
LOW	1.0 pt max	Minor defects: leading/trailing whitespace, pattern outlier clusters, mixed date formats.

Readiness Categories

Category	Score Range	Migration Readiness Action
Ready	**90.0 – 100.0**	Data meets high quality standards. Migration may proceed immediately.
Ready with Minor Cleansing	**75.0 – 89.9**	Minor formatting/whitespace defects. Can be cleansed via automated ETL transforms.
Needs Cleansing	**50.0 – 74.9**	Significant defects detected. Requires source-level data correction before loading.
Not Ready	**Below 50.0**	Critical blockers or integrity failures. Migration must be halted until resolved.

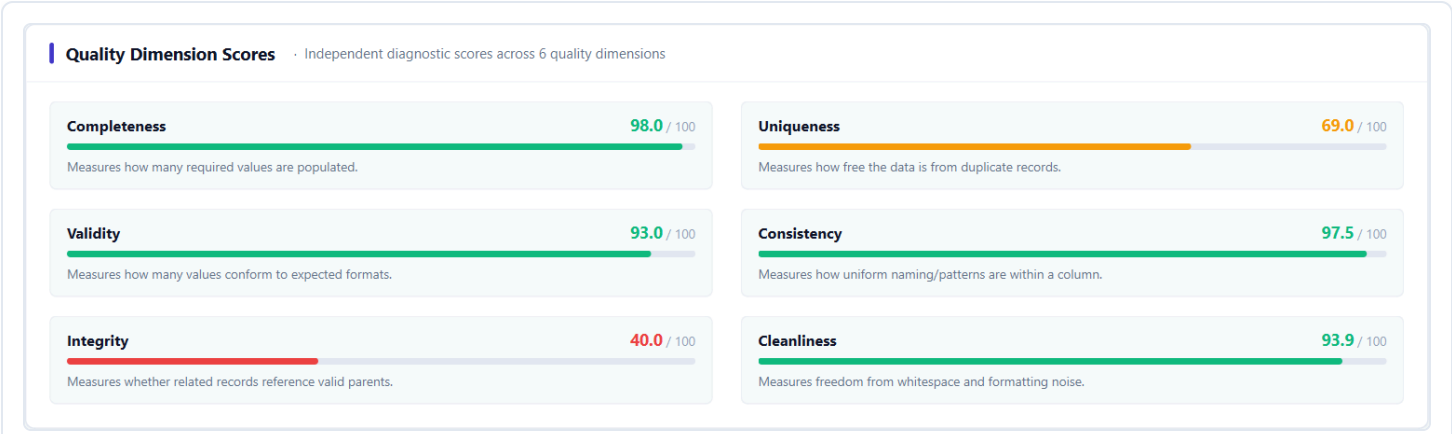
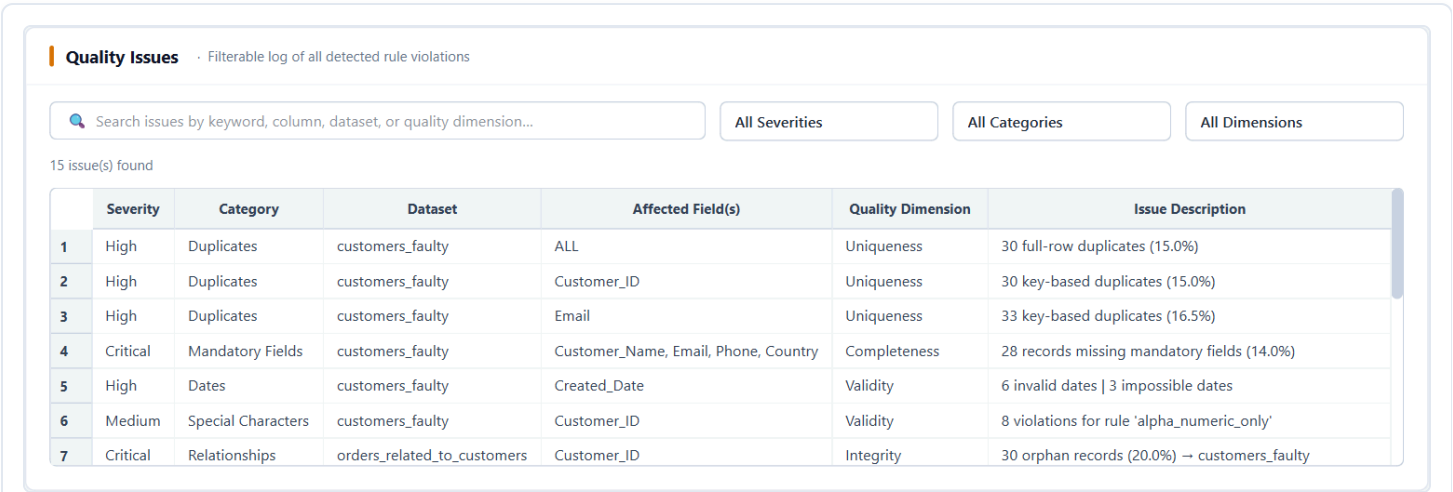


Figure 7.2: Diagnostic health progress bars across the six dimensions.

> ⚠️ **Diagnostic Dimensions Notice**
>
> The six dimension bars (Completeness, Uniqueness, Validity, Consistency, Integrity, Cleanliness) are independent diagnostic indicators. They provide health insights but do not dilute or distort the overall issue-driven readiness score.

8. Understanding Results & Issues

Detected quality violations are displayed in the **Quality Issues** table with real-time text search and multi-criteria dropdown filters:



	Severity	Category	Dataset	Affected Field(s)	Quality Dimension	Issue Description
1	High	Duplicates	customers_faulty	ALL	Uniqueness	30 full-row duplicates (15.0%)
2	High	Duplicates	customers_faulty	Customer_ID	Uniqueness	30 key-based duplicates (15.0%)
3	High	Duplicates	customers_faulty	Email	Uniqueness	33 key-based duplicates (16.5%)
4	Critical	Mandatory Fields	customers_faulty	Customer_Name, Email, Phone, Country	Completeness	28 records missing mandatory fields (14.0%)
5	High	Dates	customers_faulty	Created_Date	Validity	6 invalid dates | 3 impossible dates
6	Medium	Special Characters	customers_faulty	Customer_ID	Validity	8 violations for rule 'alpha_numeric_only'
7	Critical	Relationships	orders_related_to_customers	Customer_ID	Integrity	30 orphan records (20.0%) → customers_faulty

Figure 8.1: Searchable and filterable Quality Issues table.

Table Columns Explained

- **Severity:** Defect impact level (Critical , High , Medium , Low).
- **Category:** Check module (e.g. *Mandatory Fields*, *Duplicates*, *Dates*).
- **Dataset:** The specific source dataset where the defect occurred.
- **Affected Field(s):** The column or key set containing the defect.

- **Quality Dimension:** The related quality dimension category.
- **Issue Description:** Summary describing the nature of the violation and volume affected.

Three Types of Findings

Finding Type	What It Represents	Example
Aggregate / Dataset-Level	High-level structural metrics across an entire column or dataset.	*"Country column has 24.5% overall null rate across 5,000 rows."*
Row-Level Issue	A specific defect occurring on a concrete source record.	*"Row 15: Email contains invalid character '$' in value 'john$doe@company.com'."*
Relationship / Orphan	A child foreign key that points to a non-existent parent primary key.	*"Row 42: Order references Customer_ID 'CUST-999', which does not exist in Customers."*

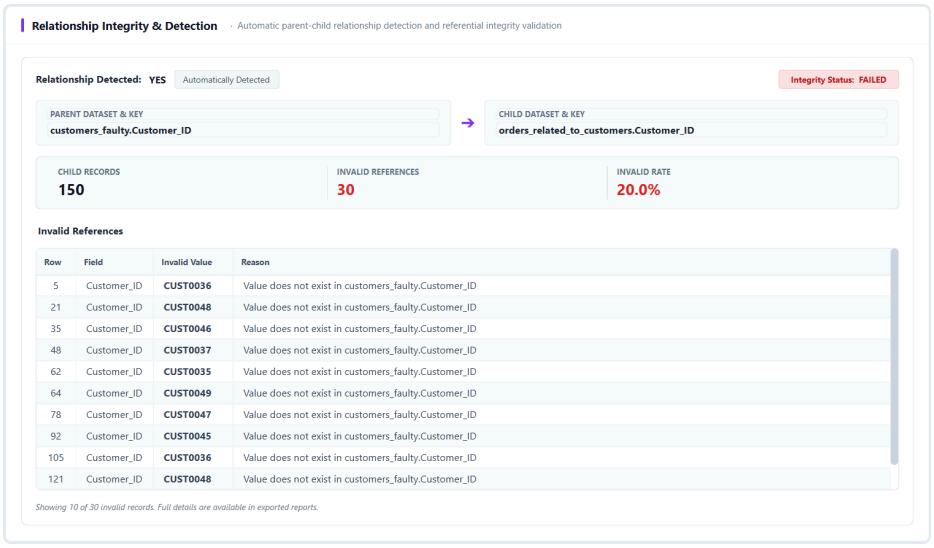


Figure 8.2: Referential integrity findings displaying parent/child linkage and exact orphan records.

9. Assessment Reports

Data Quality Scanner generates executive-ready reports saved under `reports/<dataset_name>_data_quality_report.<ext>`:

1. Multi-Sheet Excel Workbook (.xlsx)

The primary deliverable for data engineering and remediation teams. Formatted with professional styling and freeze panes across up to 7 dedicated sheets, depending on the assessment results. Relationship details are included when relationship results are available:

- **1. Summary:** Executive dashboard, scan metadata, readiness score, dimension health summary, and top findings.
- **2. Dataset Details:** Row counts, column counts, null percentages, duplicate row metrics, and file attributes.
- **3. Quality Issue Summary:** Roll-up table showing problem counts and pass/fail status per check category.

- **4. Issue Details: The core remediation table.** Contains an AutoFilter on row 1 (columns A to H): `Row` | `Dataset` | `Category` | `Severity` | `Field / Column` | `Actual Value` | `Issue Description` | `Context / Detail`
  Aggregate findings are placed in a clearly separated section below the row-level table outside the AutoFilter.
- **5. Quality Dimensions:** Independent diagnostic dimension scores (/100) and explanations for the observed quality health.
- **6. Relationships (when applicable):** Parent/child linkage table and the **Orphan Reference Details** table (listing exact child row numbers and missing key values).
- **7. Recommendations:** Prioritized remediation tasks sorted by severity with affected columns.

2. Standalone HTML Report (.html)

A self-contained, responsive web report ideal for management review and project sign-off. Includes interactive KPI cards, collapsible findings sections, and SVG score meters. Requires no internet connection.

3. Structured JSON Export (.json)

Machine-readable export containing full scan metadata, scores, and flat defect arrays with row numbers for ingestion into automated migration pipelines or data catalogs.

10. Headless Batch / CLI Mode

Basic Command Syntax

```
python batch_runner.py [options] DATASET [DATASET ...]
```

Common Command Examples

```
# 1. Quick single dataset scan (interactive export format prompt)
python batch_runner.py samples/customers_clean_100pct.csv

# 2. Multi-dataset scan with automated Excel and HTML reports
python batch_runner.py samples/customers_clean_100pct.csv samples/orders_related_to_customers.csv --
excel --html

# 3. Scan dataset using specific business profile and custom output folder
python batch_runner.py samples/customers_faulty.csv --profile Customer_Master --output reports/ --
excel --json

# 4. Analyze Excel spreadsheet with worksheet selection (by index or sheet name)
python batch_runner.py path/to/data.xlsx --sheet 0 --excel

# 5. Non-interactive CI/CD build run (quiet console, reports disabled, relies on exit code)
python batch_runner.py samples/customers_clean_100pct.csv --quiet --no-reports --no-db
```

Command-Line Options Reference

Option / Flag	Description
`DATASET...`	One or more CSV or Excel (`.xlsx` / `.xls`) files to analyze (positional).
`--config, -c CONFIG`	Path to YAML configuration file (defaults to `config/config.yaml` if present).
`--profile, -p PROFILE`	Mandatory fields profile name defined in configuration (e.g. `Customer_Master`).
`--output, -o OUTPUT`	Directory to save generated assessment reports (default: `reports`).
`--db DB`	Custom SQLite database path for scan persistence (default: `data/scanner.db`).
`--no-db, --skip-db`	Disable persisting scan findings to SQLite database.
`--sheet SHEET`	Excel sheet name or 0-based index to load when analyzing Excel files (default: 0 / first sheet).
`--html`	Generate standalone HTML assessment report.
`--excel`	Generate multi-sheet Excel (`.xlsx`) assessment report.
`--json`	Generate machine-readable JSON structured report.
`--no-reports`	Disable file-based report export entirely.
`--quiet, -q`	Suppress human-readable console progress and summary output. (Skips reports if format flags are omitted).
`--version, -v`	Show program version number and exit.
`-h, --help`	Show command-line help message and exit.

Process Exit Codes

Exit Code	Status	Meaning
`0`	**Success**	Analysis completed successfully (regardless of data quality findings).
`1`	**CLI Argument Error**	Invalid command-line argument syntax or missing dataset argument.
`2`	**Data / Config Error**	Dataset file not found, unreadable, or invalid YAML configuration syntax.
`3`	**Runtime Error**	Unexpected application or pipeline error occurred during execution.

11. Configuration Management

The scanner cleanly separates end-user business rules from administrator technical parameters:

1. User Business Rules (`config/config.yaml`)

This file is intended for data analysts and migration specialists to define dataset-specific expectations:

- **Mandatory Fields:** Columns that must not contain empty or missing values.
- **Duplicate Keys:** Columns that uniquely identify a record (e.g. `Customer_ID`).
- **Field Character Format Rules:** Maps columns to built-in character validation presets (`email_format` , `alpha_numeric_only` , `numeric_only` , etc.).
- **Configured Relationships:** Explicit parent-child links between datasets for foreign-key validation.

2. Administrator Parameters (`config/admin_config.yaml`)

> 🔒 **Administrator Settings Notice**
> Administrator settings should only be changed by authorized users. Normal users do not need to modify this file.

Controls technical engine strictness, system limits, and algorithm parameters:

- **Active Quality Checks:** Master enable/disable toggles for check modules (`checks.enabled`).
- **Scoring Weights & Readiness Thresholds:** Penalty deductions and readiness category score boundaries (`scoring`).
- **Duplicate & Null Severity Thresholds:** Dataset percentage cutoffs triggering High vs. Medium severity (`duplicate_rules` , `null_rules`).
- **Anomaly Symbols:** Global characters flagged across text fields (`special_chars.anomaly_characters` : `$` , `@` , `#` , `%` , `&` , `*`).
- **Accepted Date Formats & Column Hints:** Canonical date parsing formats and column keyword hints (`date_rules`). Note that date rules are configured here, not in `config.yaml` .
- **Pattern Rules & Warning Limits:** Pattern clustering parameters (`pattern_rules`) and memory warning limits for large datasets (`row_count_warning_threshold`).

12. Automatic Rule Inference

When running without pre-configured rules, the scanner profiles column contents to infer high-confidence rules automatically:

- **Mandatory Field Candidates:** Identified through semantic naming patterns (e.g. `_id` , `_key` , `code` , `email`) and high fill rates.
- **Record Identity Keys:** Identified through 100% uniqueness, structural compactness, and key-like naming tokens.
- **User Confirmation:** In Desktop GUI mode, proposed mandatory fields are displayed in the confirmation dialog for user approval before activation.
- **Preservation of Explicit Rules:** Configured business rules in `config/config.yaml` always take precedence and are never overwritten by inferred rules.

> 💡 **Deterministic Heuristic Inference**
> Rule inference is driven by transparent statistical profiling and heuristic scoring rules. It does not use generative AI, machine learning models, or external cloud services.

13. Scan History & Persistence

By default, completed scans are recorded in the local SQLite database (`data/scanner.db`). Persistence can be disabled in Batch/CLI mode using `--no-db` or `--skip-db` .

- **Historical Review:** View past scan timestamps, record counts, scores, and issue volumes.

- **Interactive Reloading:** In GUI mode, double-click any historical entry to restore its findings into the dashboard.
- **Re-Export Reports:** Generate updated Excel, HTML, or JSON reports from historical scans at any time.

> 🔒 **Privacy Notice**
> Raw dataset rows are not stored in the scan-history database. The database stores only execution metadata, score summaries, and identified defect coordinates (such as row numbers and invalid key values).

14. Troubleshooting Guide

Problem Observed	Probable Cause	Recommended Solution
Application fails to launch (`No module named 'PySide6'`)	Virtual environment is inactive or dependencies are missing.	Follow the complete setup sequence: 1. Navigate to project root: `cd path/to/migration-dq-scanner` 2. Activate virtual environment: `.\.venv\Scripts\Activate.ps1` or `source .venv/bin/activate` 3. Install dependencies: `python -m pip install -r requirements.txt` 4. Launch application: `python main.py`
PowerShell script execution error	PowerShell execution policy restricts running scripts on Windows.	Run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned` in PowerShell, or activate via Windows CMD using `.venv\Scripts\activate.bat` .
Dataset fails to load (`File not found`)	Path is misspelled or file is locked by another program.	Verify the file path. In CLI mode, wrap paths containing spaces in quotation marks. Close the file if open in Excel.
Unsupported file type error	File format is not CSV or Excel.	Save your source data as `.csv` or `.xlsx` before importing.
Excel workbook will not open	File is password-protected or corrupted.	Open the file in Microsoft Excel, remove password encryption, and save as a standard `.xlsx` workbook.
Configuration Error on startup	YAML formatting error in `config.yaml` or `admin_config.yaml` .	Check YAML indentation (use spaces, not tabs). Ensure check names match valid tokens.
CLI produces no report files	`--no-reports` was passed, or `--quiet` was run without format flags.	Specify explicit format flags (e.g. `--excel --html`) when running with `--quiet` , or omit `--no-reports` .
CLI terminates with Exit Code 2	Dataset path was invalid or configuration syntax was malformed.	Inspect terminal error output for the specific file path or YAML line number that failed.
Readiness score is lower than expected	Critical defects (missing mandatory fields or orphan foreign keys) were detected.	Inspect the Quality Issues and Relationship Integrity sections to review critical severity deductions.

15. Quick-Start Checklist

Desktop GUI Quick Start (Fresh Setup & Scan)

1. **Navigate to project:** `cd path/to/migration-dq-scanner`
2. **Create virtual environment:** `python -m venv .venv` (first time only)
3. **Activate virtual environment:** `.\.venv\Scripts\Activate.ps1` or `source .venv/bin/activate`
4. **Install dependencies:** `python -m pip install -r requirements.txt`
5. **Start application:** `python main.py`
6. **Add data:** Click ➕ **Add CSV File** or ➕ **Add Excel Workbook**.
7. **Preview:** Select dataset to view first 10 rows in preview grid.
8. **Run analysis:** Click 🚀 **Run Data Quality Analysis**.
9. **Confirm inferred fields:** Review and click **Proceed** on inferred mandatory fields.
10. **Review results:** Inspect Readiness Score, dimension meters, and issues table.
11. **Export reports:** Select report formats and click 📥 **Generate & Save Reports**.

Headless Batch / CLI Quick Start

1. **Navigate:** `cd path/to/migration-dq-scanner`
2. **Activate:** Activate virtual environment (`.\.venv\Scripts\Activate.ps1` or `source .venv/bin/activate`).
3. **Execute Scan:** Run the verified batch command:

```
python batch_runner.py
samples/customers_clean_100pct.csv
samples/orders_related_to_customers.csv
--excel --html
```

4. **Review Output:** Check console summary for score and issue counts.
5. **Check Exit Code:** Verify success status code `0`:

```
# PowerShell:  $LASTEXITCODE
# Windows CMD: %ERRORLEVEL%
# Linux/macOS: echo $?
```

6. **Open Reports:** Open generated deliverables in the `reports/` directory.